

TicketHouseV2

Minuta Técnica — Depuración de Producción

Fecha: 16 de September de 2026 | Sistema: TicketHouseV2 v3.0.0

Resumen ejecutivo

Se identificaron y resolvieron dos errores críticos que impedían el procesamiento de pagos con Recurrente en TicketHouseV2. El primero afectaba directamente la producción: ningún pago procesaba correctamente porque la verificación de firma del webhook fallaba siempre. El segundo afectaba el entorno de desarrollo: el endpoint de simulación de pago rechazaba todas las órdenes con un error de constraint de base de datos.

Problema 1 — Webhook de Recurrente: firma inválida (PRODUCCIÓN)

Síntoma

Los logs de EasyPanel mostraban el siguiente error en cada intento de pago exitoso de Recurrente:

```
[webhook.recurrente] Firma inválida: Expected payload to be of type string or Buffer.
```

La consecuencia directa era que los clientes eran cobrados pero no recibían sus tickets, ya que el webhook nunca se procesaba.

Causa raíz

Recurrente utiliza **Svix** para firmar sus webhooks con HMAC-SHA256. La librería Svix requiere que el body de la petición llegue como Buffer crudo (bytes), no como objeto JavaScript parseado.

El servidor tenía este middleware para capturar el body crudo:

```
app.use('/api/webhooks/recurrente', express.raw({ type: 'application/json' }));
```

El problema es que **express.raw()** no setea la bandera interna **req._body = true** de forma confiable cuando se monta así. Esto causaba que **express.json()**, montado inmediatamente después de forma global, re-parseara el body y lo convirtiera en un objeto JavaScript, destruyendo el Buffer que Svix necesitaba para verificar la firma.

Solución aplicada

Se reemplazó **express.raw()** por un middleware personalizado que lee el stream manualmente y setea **req._body = true** explícitamente para bloquear todos los parsers siguientes:

```
app.use('/api/webhooks/recurrente', function(req, res, next) { var chunks = [];  
req.on('data', function(chunk) { chunks.push(chunk); }); req.on('end', function() {  
req.body = Buffer.concat(chunks); req._body = true; // evita que express.json() lo  
re-parsee next(); }); req.on('error', next); });
```

Archivo modificado: server/server.js

Problema 2 — Endpoint dev confirm-order: db_update_failed

Síntoma

El endpoint de simulación de pagos en desarrollo devolvía siempre el mismo error:

```
curl -X POST http://localhost:3000/api/dev/confirm-order/:orderId  
{ "error": "db_update_failed" }
```

El log del servidor mostraba: [dev.confirm-order] Error actualizando orden: new row for relation "orders" violates check constraint "orders_payment_method_check"

Causas (encadenadas)

#	Causa	Descripción
1	Migration v3 no aplicada	La migración SQL migration-v3.sql nunca se había ejecutado en Supabase. Faltaban columnas críticas.
2	Valor incorrecto en devRoutes.js	El código usaba payment_method: "recurrente_dev_sim", valor que nunca estuvo en el constraint (ni en la migración).
3	Función RPC con conflicto de tipos	La función rpc_validate_by_public_code no podía recrearse con CREATE OR REPLACE porque el tipo de dato era incompatible.

Soluciones aplicadas

1. Aplicar migration-v3.sql en Supabase SQL Editor:

```
ALTER TABLE orders DROP CONSTRAINT IF EXISTS orders_payment_method_check; ALTER TABLE  
orders ADD CONSTRAINT orders_payment_method_check CHECK (payment_method IN  
( 'paypal', 'transfer', 'complimentary', 'recurrente' )); ALTER TABLE tickets ADD COLUMN IF  
NOT EXISTS public_code TEXT UNIQUE; ALTER TABLE orders ADD COLUMN IF NOT EXISTS  
recurrente_intent_id TEXT; CREATE TABLE IF NOT EXISTS webhook_events ( svix_id TEXT  
PRIMARY KEY, processed_at TIMESTAMPTZ DEFAULT now() );
```

2. Recrear función RPC con DROP previo:

```
DROP FUNCTION IF EXISTS rpc_validate_by_public_code(text); -- luego ejecutar el CREATE  
OR REPLACE de migration-v3.sql
```

3. Corregir valor en devRoutes.js:

```
// Antes: payment_method: 'recurrente_dev_sim', // Después: payment_method:  
'recurrente',
```

Archivo modificado: server/routes/devRoutes.js

Verificación

Se confirmó el correcto funcionamiento del flujo completo en desarrollo local mediante el endpoint de simulación:

```
curl -X POST
http://localhost:3000/api/dev/confirm-order/e87aed1d-c23d-47f8-af76-2608972c7bb2
{"ok":true,"simulated":true,"orderId":"e87aed1d-...", "publicCodes":["TH-PH-597882"],
"message":"Orden confirmada (simulación dev – no fue un pago real)"}

```

Pendiente para producción

Estado	Tarea
Pendiente	git commit + push de server/server.js y server/routes/devRoutes.js
Pendiente	Redeploy en EasyPanel
Pendiente	Replay del webhook en dashboard de Recurrente para órdenes afectadas
Pendiente	Verificar/emitir tickets para orden 1ffc928e-12a3-41bf-8be3-9f79bd68bc81 (pagada sin tickets)